FACULDADE UNIFECAF

RELATÓRIO TÉCNICO: DESENVOLVIMENTO DO KERNELLAB - SIMULADOR DE GERENCIAMENTO DE CPU, MEMÓRIA E DISPOSITIVOS

JOÃO FELIPE SILVA DE SOUZA
SÃO PAULO / SP
Outubro/2026

Introdução

O objetivo deste projeto foi desenvolver o KernelLab, uma aplicação interativa que simula o gerenciamento de recursos de um Sistema Operacional, voltada para solucionar gargalos de infraestrutura da MedControl Systems. O servidor da empresa executa simultaneamente processos de diferentes pesos, como monitoramento, telemetria, relatórios, backups e um processo vital de emissão de alarme crítico. A necessidade do desenvolvimento do simulador surgiu após a empresa identificar atrasos no processamento de alarmes, aumento do tempo de resposta, disputa acirrada pela CPU e um número elevado de faltas de página na memória (page faults).

O que é o Simulador de SO e o Processamento de Dados

No contexto de sistemas multitarefas, o Sistema Operacional atua como uma ponte de comunicação e controle entre o hardware físico e as aplicações de software. Diferente do desenvolvimento de um kernel completo, optamos por construir uma aplicação em Python (utilizando o framework Streamlit) que reproduz matematicamente as decisões tomadas por um SO. O consumo e o cálculo dos dados ocorrem em tempo real: o código lê os parâmetros estruturados dos processos através de um arquivo JSON externo e processa a alocação de tempo e espaço dinamicamente. Isso torna o simulador escalável para validar diferentes cargas de trabalho sem a necessidade de refatorar a lógica do programa.

Conceito de Escalonamento e Memória Virtual

O gerenciamento de recursos lida com o ciclo de vida dos processos (Novo, Pronto, Executando, Bloqueado e Terminado). Como o sistema da MedControl possui apenas uma CPU, a criação de concorrência dinâmica é essencial para que o "Alarme Crítico" não sofra atraso excessivo (starvation) aguardando o fim de processos longos como o "Backup do sistema". Na parte de memória, aplica-se o conceito de Memória Virtual. O espaço de endereçamento é dividido em páginas e frames. Quando um processo tenta acessar uma página ausente na memória RAM, ocorre uma falta de página (page fault), obrigando o sistema a buscar os dados no disco, gerando uma latência que pode paralisar sistemas médicos críticos.

Explicação das Funções Técnicas e Algoritmos Utilizados

O código desenvolvido faz uso intensivo de lógicas de escalonamento clássicas para comparar a eficiência do sistema:

- **Algoritmos de CPU:** Foram implementados FCFS (First Come, First Served), SJF (Shortest Job First), Round Robin (com quantum igual a 3) e Escalonamento por Prioridade. Nos testes, o SJF obteve o menor tempo médio de espera (10,67 u.t.), porém o escalonamento por Prioridade provou-se o único capaz de tratar a criticidade do processo de Alarme com eficiência imediata.
- **Algoritmos de Substituição de Memória:** Para gerenciar os 4 frames físicos, implementou-se o FIFO, o LRU (Least Recently Used) e o algoritmo Optimal (como

limite teórico). O algoritmo FIFO ignorou o uso recente e causou 12 faltas de página, enquanto o LRU demonstrou superioridade ao causar apenas 10 faltas, por se basear na localidade temporal.

Regras de Acesso e Segurança de Dispositivos

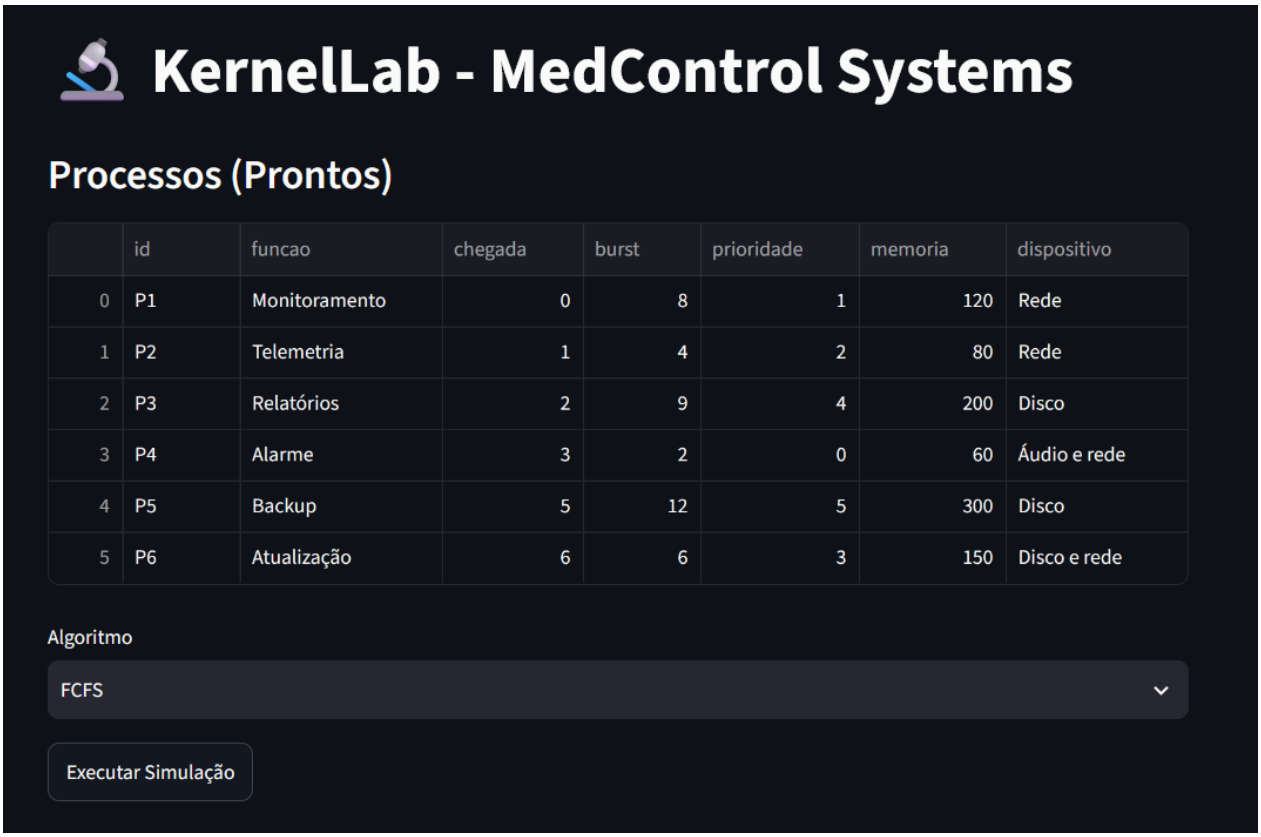
Dispositivos de I/O são recursos críticos. Se o sistema permitisse acessos livres, a concorrência poderia corromper arquivos ou interceptar pacotes de rede. Para proteger a plataforma, o simulador implementou os seguintes mecanismos:

- **Matriz de Controle de Acesso:** Relaciona processos a dispositivos com direitos explícitos de Leitura (R), Gravação (W), Execução (X), Uso (U) ou Uso Exclusivo (E). O processo de telemetria, por exemplo, é barrado se tentar acessar o disco não autorizado.
- **Mutex e Semáforos:** Quando processos concorrentes solicitam a placa de rede simultaneamente, o Mutex trava o dispositivo para um único uso, alocando os demais em uma fila de espera ordenada.

Resultados, Telas e Códigos do Projeto

Os espaços abaixo estão reservados para inserir as capturas de tela e os trechos de código solicitados, após a execução e validação da aplicação.

Dados padrão



A. Tela 1: Diagrama de Gantt e Métricas da CPU



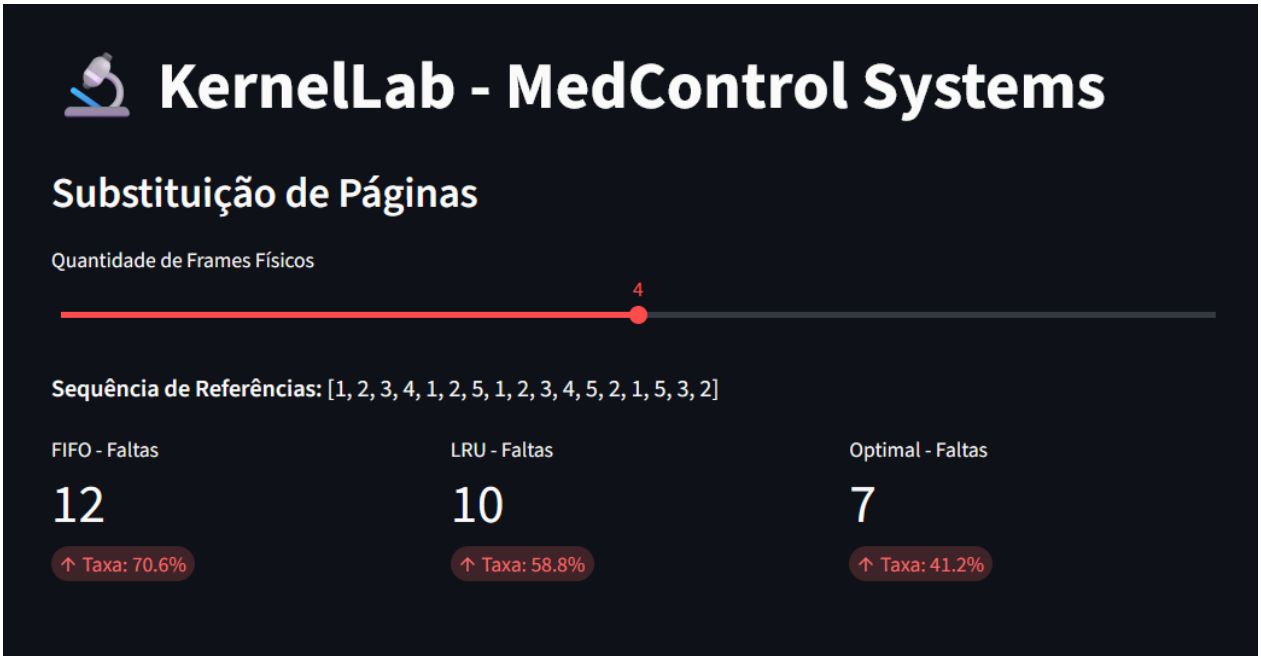
	ID	Espera	Retorno	Resposta
0	P1	0	8	0
1	P2	7	11	7
2	P3	10	19	10
3	P4	18	20	18
4	P5	18	30	18
5	P6	29	35	29

B. Código 1: Algoritmo de Escalonamento por Prioridade

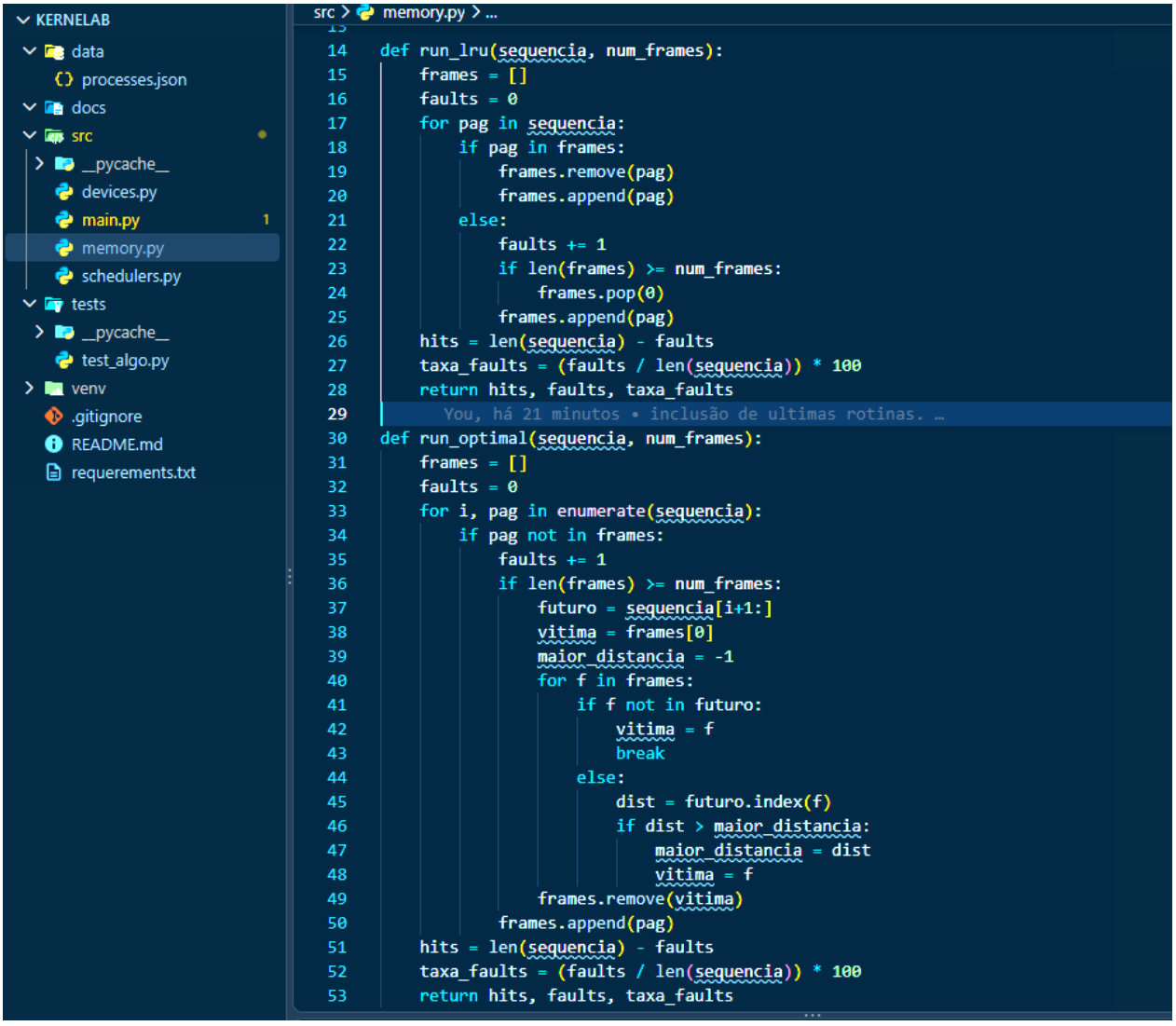


	ID	Espera	Retorno	Resposta
0	P1	0	8	0
1	P4	5	7	5
2	P2	9	13	9
3	P6	8	14	8
4	P3	18	27	18
5	P5	24	36	24

C. Tela 2: Resultados da Memória Virtual



D. Código 2: Algoritmo LRU (Least Recently Used)



E. Tela 3: Controle de Dispositivos e Mutex



	Processo	Disco	Rede	Áudio	Log
0	P1 Monitoramento	-	R, W, U	-	W
1	P2 Telemetria	-	R, W, U	-	W
2	P3 Relatórios	R, W, U	-	-	R, W
3	P4 Alarme crítico	-	W, U, E	W, U, E	W
4	P5 Backup	R, W, U, E	-	-	W
5	P6 Atualização	R, W, X, U	R, W, U	-	W

Recomendação Técnica Final

Baseado nas análises do KernelLab, a solução adequada para o cenário da MedControl consiste na adoção do Escalonamento por Prioridade Preemptivo com mecânica de aging para a CPU, assegurando que o alarme (prioridade 0) nunca sofra com bloqueios. Para a Memória Virtual, recomenda-se a política LRU por sua estabilidade e prevenção à Anomalia de Belady. Por fim, o acesso aos dispositivos deve ser governado por Negação por Padrão (default deny) integrada a controle estrito de concorrência por Mutex e filas FIFO.

Links do Projeto

Repositório GitHub Público: https://github.com/joemust0/kernelab

Vídeo Pitch (Apresentação Prática): https://youtu.be/3a4ZtIssiPw